

UNIT 2 · WEEK 2

System calls and low-level I/O

📅 Week 2 · dates to be confirmed </> Lab TD1 (solved + AI challenge) ~ 4 animations

MANUAL M2 · THEORY OF THE UNIT

System calls and low-level I/O

Core reading for unit 2 (≈ 1.5 h). Every section ends with a **Practice this** → link to the exercise or the animation that illustrates it. The figures come from real runs of the TD1 programs on the course machine (Linux 7.0, x86-64, glibc 2.43).

CONTENTS

§1	User mode and kernel mode	§4	Low-level I/O with <code>read</code> and <code>write</code>
§2	Anatomy of a system call	§5	How much a call costs
§3	Observing: <code>man</code> , <code>strace</code> , <code>objdump</code>	§6	Beyond: vDSO, <code>io_uring</code> , other architectures

§1 User mode and kernel mode

The processor has at least two **privilege levels**. On x86-64 they are called rings: the kernel runs in ring 0 (CPL 0) and programs in ring 3 (CPL 3). In user mode, instructions that touch the hardware (input/output, the page table, disabling interrupts) are forbidden: if a program tries them, the processor raises an exception and the kernel terminates it.

That boundary is the foundation of everything else: thanks to it a program cannot read another program's memory, write to the disk bypassing permissions or take over the CPU. When a program needs something only the kernel can do, **it asks**: that is a system call.

§2 Anatomy of a system call

When your program writes `write(1, "hola\n", 5)`, it actually calls a glibc function (the *wrapper*) that follows the Linux x86-64 convention:

1. It puts the **call number** in register `rax` (1 for `write`, 0 for `read`, 110 for `getppid`) and the arguments in `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`. The data does not travel in the registers: `rsi` carries the *address* of the text.
2. It executes the `syscall` instruction: the CPU switches to kernel mode and jumps to a fixed address the kernel configured at boot (`entry_SYSCALL_64`). The program cannot choose where to jump.
3. The kernel uses `rax` as an index into `sys_call_table` and runs the handler (`ksys_write`), which validates the descriptor, copies the data from program memory (`copy_from_user`) and calls the device driver.

4. The result comes back in `rax` and the `sysret` instruction returns the CPU to user mode.
5. If `rax` is between `-4095` and `-1`, the wrapper treats it as an error: it stores the code in `errno` and returns `-1`. That is why the pattern of the whole course is `exit_if(rc == -1, "...")`, which prints the error with `perror`.

Real output

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-gnu/libc.so.6 | grep -A4 -E '^[0-9a-f]+ <getppid@>'
000000000000ff710 <getppid@GLIBC_2.2.5>:
    ff710:    endbr64
    ff714:    mov     $0x6e,%eax
    ff719:    syscall
    ff71b:    ret
```

Practice this → [animation A2.1 · Journey of a system call](#)

§3 Observing: `man`, `strace`, `objdump`

Three tools to investigate any program without its source code:

- `man 2 write` documents the system call; `man 3 printf`, the library function; `man 3p`, the POSIX standard.
- `strace ./program` shows every system call with its arguments and result; `-f` follows the children, `-r` adds the time between calls and `-c` makes a summary.
- `objdump -d` disassembles: that is how you see the `syscall` instruction inside `libc`.

A surprise when using `strace`: TD1's `echo` makes about **30 calls before reaching** `main`. They belong to the dynamic loader, which opens `libc.so.6`, maps it into memory with `mmap` and protects its pages with `mprotect`.

Practice this → [animation A2.2 · strace, live](#)

§4 Low-level I/O with `read` and `write`

Every process is born with three open **file descriptors**: 0 (standard input), 1 (standard output) and 2 (standard error). The shell decides what they point to: `< file`, `> file`, `2>&1` or a pipe. The program does not know: it only uses the numbers.

CALL	RETURNS	WATCH OUT FOR
<code>read(fd, buf, n)</code>	bytes read (1...n), 0 at end of file, -1 on error	it may return <i>fewer</i> than <code>n</code> (pipe, terminal, network, signal)
<code>write(fd, buf, n)</code>	bytes written (1...n), -1 on error	partial write (full pipe or socket); <code>EINTR</code> if a signal interrupts it

That is why the course programs repeat: `read_exact` reads until the request is complete and `write_all` writes until everything is out. Also, `read` and `write` copy **bytes**, not text: writing a `long` copies its 8 bytes in machine order (*little-endian* on x86-64), and writing a structure also copies its alignment padding (24 bytes for `struct nopad`).

Real output

```
$ ./write_number | od -A d -t x1
00000000 05 00 00 00 00 00 00 00 08 00 00 00 00 00 00
0000016
```



Classic bug: advancing the pointer by what is left

The reading example of the original slides (B25, slide 51) did `length = length - byte; buf = buf + length;` . With a regular file it works —the first read brings everything and `length` is 0—, but with a pipe or a socket it leaves holes and writes **outside the buffer**. The right way is `buf += byte`, retrying on `EINTR` and stopping on 0. Watch it step by step in A2.4 and try it with `cat file | ./read_struct`.

Practice this → TD1, exercises 3 to 7

§5 How much a call costs

Every round trip to the kernel has a fixed cost: switching modes, saving and restoring registers, validating arguments. TD1's `bench_read` program measures it by copying 8 MiB to `/dev/null` with buffers from 1 byte to 1 MiB. These are the real results on the course machine:

Real output

```
$ ./bench_read      # 8 MiB file, buffers of 1 B to 1 MiB
buffer_bytes,read_calls,seconds,MB_per_s,ns_per_iteration
1,8388609,6.4984,1.3,775
4,2097153,1.5997,5.2,763
16,524289,0.3952,21.2,754
64,131073,0.0998,84.1,761
256,32769,0.0260,322.6,794
1024,8193,0.0075,1121.4,913
4096,2049,0.0025,3351.2,1222
16384,513,0.0010,8319.5,1966
65536,129,0.0008,10834.7,6002
262144,33,0.0007,11832.2,21484
1048576,9,0.0008,10287.9,90598
```

With 1 byte per call there are 8.4 million iterations and the copy takes seconds; with 64 KiB, 129 iterations and less than a millisecond. Fitting the model $time = calls \times c_0 + bytes \times c_1$ to the measurements gives a toll of about **0.7 μ s per iteration** (`read` + `write`) and less than 0.1 ns per copied byte. Practical conclusion: use buffers of a few KiB or more (the C library does it for you with `printf` / `fwrite`, unit 4) and never read one byte at a time in a loop.

Practice this → **animation A2.3 · The cost of a system call**

§6 Beyond: vDSO, `io_uring` and other architectures

- **vDSO.** Some very frequent "calls", such as `clock_gettime`, do not enter the kernel: Linux maps into every process a small library (vDSO) that reads the time from a shared page. That is why they do not show up in `strace`.
- **`io_uring`** (Linux 5.1, 2019). For programs that do a lot of I/O: the program and the kernel share two queues in memory; hundreds of operations can be submitted with a single call, or none. The same idea as the big buffer, applied to the calls.
- **Other architectures.** On 64-bit ARM the instruction is `svc #0` and the number goes in `x8`; on RISC-V it is `ecall` with the number in `a7`. The numbers change between architectures; the idea does not.
- **Observability with eBPF.** Tools like `bpftrace` count calls across the whole system at a much lower cost than `strace`, which stops the process at every call.